컴퓨터네트워크 실습 2–7주차 · 오픈북 핵심 요약

ASD-STE100 원칙으로 썼다. 한 문장에 한 내용만 쓴다. 문장은 짧게 쓴다. 능동형으로 쓴다. 같은 대상에는 항상 같은 용어를 쓴다. 코드는 이 저장소에서 `--verify`와 `bench.py`를 통과한 코드다. "측정"이라고 쓴 수치는 `wNN/out/`의 결과다.

0 · 이 책 사용법 HOW TO USE

1. 아래 가나다·ABC 색인에서 키워드를 찾는다. 오른쪽 절 코드(예: **T4**)로 간다.
2. 색 하나가 한 주차다. 모든 소제목 앞에 절 코드가 있다.
3. 코드 문제는 먼저 **C절**을 본다. 모든 주차의 코드 패턴과 함정이 있다.
4. "왜" 문제는 각 주차의 **왜** 항목을 본다. 시험식 답이 정리되어 있다.

코드	주차 · 주제	코드	주차 · 주제
A	숫자·공식 한 장	R	6주차 라우팅: Dijkstra, DV, OSPF, BGP, ICMP
H	2주차 HTTP, 에이전트 실습	E	7주차 이더넷, ARP, 스위치, MAC
D	3주차 DNS, CDN, DASH	C	코드 패턴과 함정 (전 주차)
T	4주차 TCP: rdt, 핸드셰이크, 혼잡 제어	X	명령어: dig, tshark, ip, arp, traceroute
I	5주차 IP, CIDR, LPM, NAT, DHCP		

IDX 키워드 색인 (가나다 → ABC)

가장 긴 접두사 (LPM)	I3, C6, C7	에이징 (aging)	E3, C10	DIR-24-8	I6, C7
거리 벡터 (DV, Bellman-Ford)	R2	위임 (delegation, referral)	D1, D3	Fast retransmit (중복 ACK 3개)	T5
게이트웨이	I4, E2	이용률 (utilization)	A, T1	Glue 레코드	D1, D3
권한 서버 (authoritative)	D1	점진 증가·곱셈 감소 (AIMD)	T5, T6, C5	Go-Back-N	T1
권한 응답 플래그 (AA)	D3	주소 변환 (NAT)	I4	heapq	C0, C8
누적 ACK	T1, T3	중복 패킷	T1, C4	Hello / Dead 간격	R3, R6
다음 홉 / 첫 홉	R1, C8	증분 SPF	R7, C9	HOL 블로킹	I1, H1
대역폭·지연 곱 (BDP)	A, T5	최단 경로 (Dijkstra)	R1, C8	HTTP 지속 연결·RTT	H1
동일 비용 다중 경로 (ECMP)	R3, R6	캐시·TTL	D4, C3	ICMP / traceroute	R5
동점 규칙 (tie-break)	R1, C8	타임아웃 계산	A2, T2	IPv4 / IPv6 헤더	I1
링크 상태 (link state)	R1, R3	포트 보안 (port security)	E5	MAC 주소, 프레임	E1
만료된 답 (stale)	D4	플러딩 (스위치)	E3, E5	MAC flooding 공격	E5
무한 카운트 (count to infinity)	R2	플러딩 (LSA)	R3	MinLSArrival	R6
반복 질의 / 재귀 질의	D1, C2	핸드셰이크 (3-way)	T3	MSS / window scale / SACK	T3
브로드캐스트 주소	I2, E1	혼잡 붕괴	T6	NS 레코드	D2
비트 마스크·시프트	I2, C1	혼잡 윈도우 (cwnd)	T4, T5	OSPF	R3, R6
비트 트라이 (trie)	I6, C7	ARP 요청·응답	E2	Poisoned reverse	R2
사설 주소 (RFC 1918)	I4	BGP	R4	Slow start / ssthresh	T5
서브넷 (CIDR)	I2, C6	CGNAT 100.64/10	I4	Stop-and-wait	T1
선택적 반복 (SR)	T1, C4	CNAME 체인	D2, D5	STP / VLAN	E4
수신 윈도우 (rwnd)	T4	Count-min sketch	E5, C11	TTL (DNS와 IP는 다름)	D4, I1
순서 번호 초기값 (ISN)	T3	CRC / 패리티	E4	UDP 53 / TCP 53	D3
스위치 학습	E3, C10	CSMA/CD, 백오프	E4	/31, /32 (RFC 3021)	I2
애니캐스트 (anycast)	D5, R5	DASH	D5	0.0.0.0/0 기본 경로	I3
		DHCP DORA	I5		

A · 숫자와 공식

한 장 요약

A1 지연과 처리량

항목	공식
전송 지연	$d_{trans} = L / R$
전파 지연	d_{prop} = 거리 / 속도 ($\approx 2\times10^8$ m/s)
노드 지연	$d_{proc} + d_{queue} + d_{trans} + d_{prop}$
트래픽 강도	La / R. 1보다 작게 유지한다.
Stop-and-wait 이용률	U = (L/R) / (RTT + L/R)
파이프라인 (N개)	U = N·(L/R) / (RTT + L/R)
BDP (파이프 크기)	R × RTT
TCP 처리량	≈ cwnd / RTT. AIMD 평균 ≈ 0.75·W / RTT
송신 한도	LastByteSent − LastByteAcked ≤ min(cwnd, rwnd)
스케일 적용 윈도우	rwnd = Window 필드 × 2^{shift}

A2 TCP 타이머 (RFC 6298)

```
EstimatedRTT = 0.875·EstimatedRTT + 0.125·SampleRTT
DevRTT       = 0.75·DevRTT + 0.25·|SampleRTT − EstimatedRTT|
TimeoutInterval = EstimatedRTT + 4·DevRTT   (초기값 1초, 타임아웃마다 2배)
```

A3 서브넷 계산

항목	공식
마스크	(0xFFFFFFFF << (32−p)) & 0xFFFFFFFF
네트워크 주소	addr & mask
브로드캐스트	net | (~mask & 0xFFFFFFFF)
주소 개수	2^{32-p}
호스트 개수	$2^{32-p} - 2$ ($p \le 30$)
/31, /32	호스트 2개, 1개. 브로드캐스트 없음.

A4 접두사 표

/p	마스크	주소 수	호스트
/8	255.0.0.0	16,777,216	16,777,214
/12	255.240.0.0	1,048,576	…
/16	255.255.0.0	65,536	65,534
/20	255.255.240.0	4,096	4,094
/22	255.255.252.0	1,024	1,022
/24	255.255.255.0	256	254
/25	.128	128	126
/26	.192	64	62
/27	.224	32	30
/28	.240	16	14
/29	.248	8	6
/30	.252	4	2

A5 포트·번호

DNS	UDP/TCP 53	DHCP	UDP 67 서버, 68 클라이언트
HTTP/HTTPS	80 / 443	QUIC	UDP 443
IP 프로토콜	1 ICMP, 6 TCP, 17 UDP, 89 OSPF	BGP	TCP 179
EtherType	0x0800 IPv4, 0x0806 ARP, 0x86DD IPv6, 0x8100 VLAN	ARP opcode	1 요청, 2 응답
헤더 크기	IPv4 20 B, IPv6 40 B, TCP 20 B, UDP 8 B, DNS 12 B	이더넷	페이로드 46–1500 B, CRC 4 B
ICMP	0/8 echo, 3 도달 불가, 11 TTL 초과	OSPF 타이머	Hello 10초, Dead 40초
스위치 에이징	300초	루트 서버	이름 13개 (a–m)

A6 실습 bench 결과 (측정)

주	baseline	내 결과
3	upstream 325, stale 266	275, stale 0 (= 하한)
4	goodput 986.8, 손실 37.4%, 큐 8.8	946.2 (96%), 2.0%, 큐 4.1
5	초당 1,559회 조회	초당 410만 회, 2640배
6	SPF 1001회	177회 (82% 절감)
7	낭비 전달 323,380	159,712 (−51%)

H · 2주차 · HTTP와 에이전트 실습 § 2.1–2.2

H1 HTTP 핵심

- HTTP는 클라이언트–서버 프로토콜이다. TCP 80을 쓴다(HTTPS는 443).
- HTTP는 **무상태(stateless)**다. 서버는 클라이언트 상태를 보관하지 않는다. 쿠키가 상태를 더한다.
- **비지속 연결**: 객체마다 TCP 연결을 하나 연다. 객체 하나에 2·RTT + 전송 시간이 든다.
- **지속 연결**(HTTP/1.1 기본): 연결 하나로 여러 객체를 보낸다. 첫 객체는 2·RTT, 다음 객체는 약 1·RTT다.
- HTTP/1.1은 객체를 차례대로 보낸다. 큰 객체가 작은 객체를 막는다(**HOL 블로킹**).
- HTTP/2는 객체를 **프레임**으로 나누고 한 TCP 연결에 섞어 보낸다. 그래도 TCP 손실은 모든 스트림을 막는다.
- HTTP/3는 UDP 위의 **QUIC**을 쓴다. 스트림마다 신뢰성을 따로 처리한다. 한 스트림의 손실이 다른 스트림을 막지 않는다. 전송·TLS 핸드셰이크를 합친다(1 RTT, 재접속 0-RTT).
- 웹 캐시(프록시)는 응답 시간과 접속 링크 부하를 줄인다. 조건부 GET은 `If-Modified-Since`를 보낸다. 바뀌지 않았으면 `304 Not Modified`가 온다.

H2 상태 코드와 메서드

200 OK	301 Moved Permanently
304 Not Modified	400 Bad Request
404 Not Found	505 HTTP Version Not Supported

메서드: GET, POST, HEAD, PUT, DELETE. 요청 줄은 `GET /path HTTP/1.1`이고, 다음에 `Host:` 헤더가 온다. 빈 줄이 헤더의 끝이다.

H3 에이전트 실습 요점

- Task 1: 에이전트가 자료(HTTP/QUIC의 RFC, 논문)를 모은다.
- Task 2: 주장을 하나씩 직접 확인한다. 틀린 주장의 수를 센다.
- Task 3: 작업 설정(AGENTS.md)을 개선한다. 다시 센다.
- 요점: "완성되어 보인다"와 "맞다"는 다르다. 모든 주장을 원문과 대조한다.

D · 3주차 · DNS, CDN

§ 2.4.2–2.4.3, § 2.5

D1 계층과 이름 풀이

- DNS는 분산된 계층형 데이터베이스다. 각 단계는 아래 존(zone)을 **위임**한다.
- 단계: **루트** → **TLD**(.com, .kr) → **권한 서버**(korea.ac.kr).
- 루트는 호스트 주소를 모른다. 루트는 TLD 서버만 안다. 그래서 **위임(referral)**을 돌려준다.
- 호스트는 로컬 리졸버에 **재귀 질의 1번**을 보낸다(RD=1).
- 리졸버는 루트, TLD, 권한 서버에 **반복 질의**를 보낸다(RD=0, `+norecurse`).
- 측정: `www.korea.ac.kr`은 서버 3곳(루트 → .kr → 학교)에 물었다.
- **Glue** = NS 이름의 A 레코드다. Additional 섹션에 온다. NS 이름이 위임된 존 안에 있으면 부모가 glue를 준다.
- **Glue 없음**: NS 이름이 다른 도메인에 있다. 리졸버는 NS 이름을 루트부터 먼저 풀어야 한다. 즉 두 번째 탐색이다. 측정: `www.nytimes.com`에서 질의 +3건(총 13건).
- 실습 요구 R1–R6: (주소, 경로)를 돌려준다. 루트에서 시작한다. 재귀 질의를 쓰지 않는다. glue 없음 처리(R3). 응답 없으면 다음 서버(R4). CNAME이면 처음부터 다시(R5). 깊이 제한(R6).

D2 리소스 레코드 (name, value, type, TTL)

Type	Name → Value
A / AAAA	호스트 이름 → IPv4 / IPv6
NS	도메인 → 권한 서버의 호스트 이름
CNAME	별칭 → 정식 이름(canonical)
MX	도메인 → 메일 서버 이름
SOA, PTR, TXT	존 정보; IP → 이름; 텍스트

D3 메시지 형식 (질의와 응답은 같은 형식)

필드	뜻
ID (16비트)	응답을 질의와 짝짓는다.
QR	0 질의, 1 응답
AA	1 = 권한 서버가 직접 답했다.
RD / RA	재귀 요청 / 재귀 가능
TC	잘림 → TCP 53으로 다시 묻는다.
RCODE	0 NOERROR, 3 NXDOMAIN
개수	QD, AN, NS(authority), AR(additional)

	위임 응답	답변 응답
Answer	0개	A 레코드
Authority	NS 레코드	(NS가 올 수 있음)
Additional	glue A	—
AA	0	1

큰 응답은 NS와 glue가 많다(IPv6 glue 포함, EDNS0로 512 B 초과).

D4 캐시와 TTL (Task 3)

- TTL은 권한 서버가 정한 "이 시간까지만 믿어도 된다"는 값이다.
- 올바른 캐시는 레코드를 **그 레코드의 TTL 동안만** 보관한다. `만료 시각 = now + ttl`을 저장한다.
- baseline 문제 1(정확성): 수명을 60초로 고정한다. 짧은 TTL 레코드를 만료 뒤에도 준다 → **stale 266건**.
- baseline 문제 2(성능): 긴 TTL 레코드도 60초 뒤 버린다 → 불필요한 상류 질의. 리스트 선형 탐색도 느리다.
- 원인은 같다. baseline이 TTL을 버린다.
- **하한**: 만료 뒤 첫 질의는 반드시 상류로 간다. 어떤 올바른 캐시도 이것을 피할 수 없다. 하한 = 275 = 내 결과.
- 최악의 레코드: `www.microsoft.com`. TTL 20초이고 질의가 가장 많다(322건 중 189건 stale).

D5 CDN, 유도(steering), DASH (Task 2)

- 사이트는 **CNAME 체인**으로 CDN을 가리킨다(예: → `akamaiedge.net`).
- 어떤 CDN은 **애니캐스트**를 쓴다. IP 하나, 위치 여러 곳, CNAME 없음.
- Netflix는 netflix.com 안에서 끝난다. 자체 CDN을 운영한다.
- "마지막 두 레이블 비교" 규칙은 `www.wikipedia.org` → `wikimedia.org`에서 틀린다. 존은 다르지만 소유자는 같다. CNAME은 이름을 보여 줄 뿐 소유자를 보여 주지 않는다.
- 측정: 12곳 중 10곳이 CNAME으로 CDN에 연결. 10곳 중 8곳은 리졸버를 바꾸면 답이 바뀜. 3곳(Akamai)은 네트워크를 바꾸면 바뀜.
- DASH: 서버는 영상을 여러 비트레이트의 청크로 나눈다. 매니페스트가 URL을 나열한다. 클라이언트는 측정한 대역폭으로 청크마다 비트레이트를 고른다.

D6 왜 — 시험식 답

질문	답
루트는 왜 주소를 주지 않는가?	루트는 TLD 위임만 안다. 각 존의 데이터는 그 존의 소유자가 보관한다. 이렇게 부하와 관리를 나눈다.
DNS는 왜 UDP인가?	짧은 질문 하나, 짧은 답 하나다. 핸드셰이크 비용이 없다. TCP는 큰 응답(TC=1)과 존 전송에 쓴다.
왜 캐시하는가?	루트·TLD 질의를 없애고 지연을 줄인다. TTL은 답이 얼마나 오래되어도 되는지 제한한다.
리졸버마다 답이 왜 다른가?	CDN은 클라이언트 IP가 아니라 리졸버 IP를 본다. 리졸버에 가까운 복제 서버를 고른다.

T1 신뢰적 데이터 전송 (rdt)

	Go-Back-N	Selective Repeat
ACK	누적 ACK	패킷마다 ACK
타이머	1개 (가장 오래된 것)	패킷마다 1개
타임아웃 시	base부터 전부 재전송	그 패킷만 재전송
수신기	순서가 틀리면 버림	순서가 틀려도 버퍼에 보관
윈도우	$\leq 2^k - 1$	$\leq 2^{k-1}$ (순번 공간의 절반)

- Stop-and-wait: 한 번에 패킷 하나만 보낸다. 쉽다. 느리다. RTT와 타임아웃마다 채널이 논다.
- 순서 번호로 중복과 순서를 알아낸다. ACK 번호로 어떤 데이터가 도착했는지 안다.
- **중복 데이터**: 다시 ACK한다(앞의 ACK가 손실되었을 수 있다). 다시 붙이지 않는다.
- **중복 ACK**: `acked[seq]=True`를 다시 해도 문제없다.
- 실습 채널: 손실 10%, 중복 3%, 순서 뒤바뀜. 선택: SR, 윈도우 8, 타임아웃 40스텝.
- 측정: 데이터 250패킷에 326번 전송 = 1.30배(손실 42, 중복 13).
- "받은 대로 붙이는" 수신기는 **손실**에서 먼저 깨졌다(4번째 패킷). 중복은 9번째.

T2 타임아웃

공식은 A2를 본다. 재전송한 세그먼트로는 SampleRTT를 재지 않는다(Karn). 타임아웃이 나면 간격을 2배로 한다.

T3 세그먼트, 핸드셰이크, 종료

- Seq = 데이터 첫 바이트의 번호. ACK = 다음에 받을 바이트 번호(누적).
- SYN과 FIN은 각각 순서 번호 1개를 쓴다.

```
클라이언트                          서버
SYN      seq=x              →
         ←  SYN+ACK seq=y, ack=x+1
ACK      seq=x+1, ack=y+1 →      (데이터를 실어도 됨)
종료: FIN → ACK, FIN ← ACK; 클라이언트는 TIME_WAIT(2·MSL) 대기
```

- **ISN은 무작위**이고 0이 아니다. 위조 세그먼트를 막는다. 같은 포트 쌍을 쓴 이전 연결의 옛 세그먼트가 섞이는 것을 막는다. 측정 ISN: 1111356914, 3798358670.
- SYN 옵션: **MSS**(예: 1460), **window scale**(shift), **SACK permitted**, timestamps. MSS·WS는 SYN/SYN-ACK에만 온다.
- 플래그: SYN, ACK, FIN, RST, PSH, URG (+ ECE, CWR).
- 핸드셰이크 시간 ≈ 1 RTT다. 모든 연결에 더해진다.

T4 흐름 제어 vs 혼잡 제어

- 흐름 제어는 **수신기** 버퍼를 보호한다: **rwnd**(헤더에 있음).
- 혼잡 제어는 **네트워크**를 보호한다: **cwnd**(송신자만 앎, 헤더에 없음).
- 측정: rwnd ≈ 4.6 MB(scale ×1024). 전송 중 최대 ≈ 193 KB. 그래서 제한 요인은 rwnd가 아니라 slow start 중인 cwnd였다. BDP ≈ 168 KB.

T5 TCP 혼잡 제어 (Reno)

사건	동작
시작	cwnd = 1 MSS, **slow start**
slow start 중 ACK	cwnd += 1 MSS (RTT마다 2배)
cwnd ≥ ssthresh	**혼잡 회피(CA)**로 간다
CA 중 ACK	cwnd += MSS·MSS/cwnd (RTT마다 +1 MSS)
중복 ACK 3개	fast retransmit. ssthresh = cwnd/2. cwnd = ssthresh + 3 (fast recovery)
타임아웃	ssthresh = cwnd/2. cwnd = 1 MSS. slow start.

- Tahoe: 어떤 손실이든 cwnd = 1. Reno: 중복 ACK 3개면 절반.
- AIMD는 톱니 모양을 만든다. AIMD 흐름 둘은 공평한 몫으로 수렴한다.
- CUBIC(리눅스 기본): 마지막 손실 이후 시간의 3차 함수로 윈도우를 키운다.
- 지연 기반(Vegas, BBR): RTT 증가를 큐의 신호로 쓴다.
- ECN: 라우터가 IP 헤더에 표시한다. 수신기가 ECE로 되돌려 준다.
- 처리량 ∝ 1/RTT. 측정: 캠퍼스 178.8 Mbps(RTT 7.5 ms), 핫스팟 57.8 Mbps(38.1 ms). 캠퍼스에 netem +31 ms: 210 → 85 Mbps. RTT만으로도 처리량이 준다.

T6 실습 Task 3 · 혼잡 bench

- 링크: 슬롯당 1패킷, RTT 20슬롯 → 파이프 = 20. 큐 10, tail drop → 30 넘으면 손실.
- baseline FixedWindow 64: goodput은 가장 높다. 그러나 손실 37%, 큐 8.8/10.
- **baseline이 최악인 이유**: goodput은 다른 흐름이 치르는 비용을 보여주지 않는다. 큐를 채워 모두를 지연시키고, 재전송에 용량을 버린다. 모두 이렇게 보내면 **혼잡 붕괴**(1986년)가 된다.
- 내 방식: slow start 뒤 AIMD ×0.5, RTT마다 +0.25. 손실 묶음 하나 = 감소 한 번(조용한 구간). 윈도우 15–30, 평균 24. 파이프 20보다 조금 크다.
- 감소 ×0.7: goodput 98.7%지만 손실 4.0%, 큐 5.6 → R4 불합격. 마지막 몇 %는 큐, 즉 다른 흐름의 지연으로 얻은 것이다.
- 조건: 손실 ≤ 5%, 평균 큐 ≤ 5.0. good ≥ 85%, strong ≥ 93%. 교과서 AIMD ≈ 82%.

I1 IP 헤더

- IPv4: 옵션 없이 20 B. 필드: version, header length, TOS(DSCP/ECN), total length, ID, flags(DF, MF), fragment offset(8바이트 단위), **TTL**, protocol, header checksum, src, dst.
- 라우터마다 TTL을 1 줄인다. 0이 되면 패킷을 버리고 ICMP 11을 보낸다.
- IPv6: 40 B 고정. 128비트 주소. flow label, hop limit, next header. 헤더 체크섬 없음. 라우터는 단편화하지 않는다(출발지가 PMTU 탐색으로 한다). 터널링으로 IPv6를 IPv4에 싣는다.
- 라우터: 입력 포트(조회) → 스위칭 패브릭(메모리, 버스, 크로스바) → 출력 큐. 입력 큐에서 **HOL 블로킹**. 스케줄링: FIFO, 우선순위, 라운드 로빈, WFQ.

I2 CIDR과 범위

- `a.b.c.d/p`: 앞 p비트가 네트워크, 나머지가 호스트다.
- 네트워크 주소: 호스트 비트 모두 0. 브로드캐스트: 호스트 비트 모두 1. 사용 가능 = 그 사이.
- p가 0–32 밖이면 거부한다(R1). 호스트 비트가 켜져 있으면 거부한다: `163.152.6.5/24`는 네트워크가 아니라 호스트다(R2).
- **/31**(RFC 3021, 점대점): 두 주소 모두 호스트 → `(net, net+1, None)`.
- **/32**: 호스트 하나 → `(addr, addr, None)`.

예제	첫 주소	끝 주소	브로드캐스트
192.168.0.0/24	.0.1	.0.254	.0.255
10.0.0.0/8	10.0.0.1	10.255.255.254	10.255.255.255
172.16.32.0/20	172.16.32.1	172.16.47.254	172.16.47.255
203.0.113.64/26	.65	.126	.127

손 계산법: 경계 옥텟을 찾는다(p=20 → 3번째 옥텟). 블록 크기 = 256 − 마스크 옥텟(240 → 16). 네트워크 = 값 이하인 블록의 최대 배수(32). 브로드캐스트 = 네트워크 + 블록 − 1(47).

I3 가장 긴 접두사 일치 (LPM)

- 접두사는 겹칠 수 있다. 라우터는 일치하는 것 중 **가장 긴**(가장 구체적인) 접두사를 고른다.
- `0.0.0.0/0`은 모두와 일치한다. 다른 일치가 있으면 진다.
- 예: `10.20.30.70`은 /0, /8, /16, /24, /26과 일치 → `/26 lab-rack-2`.
- 같은 네트워크·같은 길이에 다음 홉이 다르면 모호하다. 내 add()는 ValueError를 낸다.

I4 NAT와 특수 주소

사설 (RFC 1918)	10/8, 172.16/12, 192.168/16
CGNAT (RFC 6598)	100.64.0.0/10
링크 로컬	169.254/16
루프백	127/8
문서·시험용	192.0.2/24, 198.51.100/24, 203.0.113/24

- NAT는 (사설 IP, 포트)를 (공인 IP, 새 포트)로 바꾼다. **변환 표**를 보관한다. 응답은 이 표로 호스트를 찾는다.
- 바깥 호스트는 안으로 연결을 시작할 수 없다(포트 포워딩 없이는). NAT는 종단 간 주소 체계를 깬다.
- **NAT 단수 세기**: 인터페이스는 사설이고 바깥에서 본 공인 주소가 다르면 → NAT 최소 1개. 첫 NAT의 바깥 주소가 100.64/10이나 사설이면 → 2단(CGNAT). traceroute는 사설 홉을 보여 줄 뿐, 변환 위치는 보여 주지 않는다.
- 측정: 두 네트워크 모두 NAT ≥ 1. 정확한 개수는 증명하지 못했다. 네트워크를 바꾸면 사설·공인 주소 모두 바뀌었다.
- 게이트웨이는 반드시 내 서브넷 안에 있다. 호스트는 서브넷 밖으로 가는 프레임을 게이트웨이 MAC으로 보낸다.

I5 DHCP (DORA, UDP 67/68)

메시지	출발지 → 목적지	내용
Discover	0.0.0.0:68 → 255.255.255.255:67	chaddr, transaction ID
Offer	서버 → 브로드캐스트/클라이언트	yiaddr, 임대 시간, 마스크, 라우터, DNS
Request	0.0.0.0 → 255.255.255.255	고른 서버, 요청 IP
Ack	서버 → 클라이언트 (유니캐스트 가능)	임대 확정

- Discover 출발지는 **0.0.0.0**이다. 클라이언트는 아직 주소가 없다. 목적지는 **255.255.255.255**다. 클라이언트는 서버를 모른다.
- Ack는 유니캐스트일 수 있다. 서버가 이제 chaddr(MAC)와 yiaddr를 안다.
- T1 = 임대의 50%에 갱신한다(서버에 유니캐스트). T2 = 87.5%에 재바인딩한다(브로드캐스트).
- DORA 전체를 보려면 새로 접속해야 한다. 이전 임대를 먼저 지운다. 아니면 Request/Ack만 보인다.

I6 실습 Task 3 · 빠른 LPM

- baseline: 선형 탐색. 작업량 ∝ 라우트 수(5,000).
- 방법 1, 길이별 묶기: 사전 조회 최대 33번. 긴 길이부터 묻고 첫 일치에서 멈춘다.
- 방법 2, 비트 트라이: 최대 32단계. 작업량 ∝ 주소 폭. 메모리 ∝ 라우트 수.
- 내 방식: **DIR-24-8**. /24마다 배열 칸 1개(2^{24}칸, 64 MB). 조회 = 배열 읽기 1번. 시간이 일정하다. 2640배 빠르다.
- 하드웨어가 트라이를 쓰는 이유: 상한이 32단계로 고정된다. 메모리가 라우트 수에 비례한다. 단계마다 파이프라인으로 나눌 수 있다. 파이썬에서는 32번 도는 반복보다 사전·배열 한 번이 빠르다.

R · 6주차 · 라우팅 §5.2–5.4, §5.6

R1 링크 상태 (Dijkstra)

- 라우터마다 자기 링크 비용(LSA)을 플러딩한다. 모든 라우터가 같은 지도를 갖는다.
- 각 라우터가 혼자 Dijkstra를 돌린다. 결과는 경로가 아니라 **목적지마다 다음 홉 하나**다.
- 복잡도: 단순 구현 $O(n^2)$, 힙 사용 $O((n+e)\log n)$.
- 다음 홉은 반드시 **직접 이웃**이다. 출발지 자신은 표에 없다.
- 인접 관계를 경로로 쓰지 않는다: u–w 직접 = 5, u–x–y–w = 3 → w의 다음 홉은 **x**.
- 내 동점 규칙: 첫 홉 이름이 가장 작은 쪽. 실제 OSPF: 같은 비용의 다음 홉을 모두 설치(**ECMP**)하고 흐름마다 해시로 나눈다.
- 비용 = 트래픽 부하이면 경로가 진동할 수 있다.

교과서 그래프 (§5.2.1)	v	w	x	y	z
u에서 비용	2	3	1	2	4
u의 다음 홉	v	x	x	x	x
u–x 끊긴 뒤	v	v (동점 5)	v	v	v

링크: `u-v2 u-w5 u-x1 v-w3 v-x2 w-x3 w-y1 w-z5 x-y1 y-z2`

Dijkstra 손 계산: 표의 행 = 단계. N' = 확정된 노드 집합. 단계마다 D(·)가 가장 작은 노드를 N'에 넣는다. D(v) = min(D(v), D(w) + c(w,v))로 갱신한다. p(v) = 직전 노드를 적는다.

R2 거리 벡터 (Bellman-Ford)

```
D_x(y) = min (이웃 v에 대해) { c(x,v) + D_v(y) }
```

- 노드는 자기 벡터를 이웃에게만 보낸다. 비동기, 반복, 분산.
- 좋은 소식은 빨리 퍼진다. 나쁜 소식은 느리게 퍼진다: **무한 카운트**.
- **Poisoned reverse**: z가 y를 거쳐 x로 가면, z는 y에게 "$D_z(x) = \infty$"라고 알린다. 노드 3개 이상의 루프는 막지 못한다.
- LS vs DV: LS는 O(nE) 메시지, 전체 지도를 알고 각자 계산한다(견고). DV는 이웃에게만 보내고 수렴이 느리며 오류가 퍼진다.

R3 OSPF

- AS 내부 링크 상태 프로토콜. IP 위에서 바로 동작한다(프로토콜 89).
- Hello로 이웃을 찾는다(10초). Dead 간격(40초) 동안 hello가 없으면 이웃 down.
- LSA를 영역 전체에 플러딩한다. 라우터마다 SPF를 계산한다. 인증을 지원한다.
- 계층: **영역(area)**. Area 0 = 백본. 영역 경계 라우터가 요약한다.
- 관리자가 링크 비용을 정한다. ECMP를 허용한다.

R4 BGP (AS 간)

- AS 사이는 eBGP, AS 안은 iBGP. TCP 179를 쓴다. 경로 벡터 방식.
- 경로 = 접두사 + 속성. **AS-PATH**(루프 감지), **NEXT-HOP**.
- 선택 순서: local preference(정책) → 가장 짧은 AS-PATH → 가장 가까운 NEXT-HOP(**hot potato**) → 기타.
- 정책: AS는 고객이 아닌 쪽의 통과 트래픽을 실어 주지 않는다.
- AS 내부는 성능을 중시한다. AS 간은 정책을 중시한다. 계층 구조로 규모를 감당한다.
- SDN: 중앙 컨트롤러가 흐름 표(일치 + 동작, OpenFlow)를 계산한다.

R5 ICMP와 traceroute

- traceroute는 TTL = 1, 2, 3 …인 탐침을 보낸다. TTL이 0이 된 라우터가 ICMP 11(TTL 초과)을 보낸다.
- 목적지 호스트는 ICMP 3/3 포트 도달 불가(UDP)나 echo reply를 보낸다. 그러면 traceroute가 멈춘다.
- `* * *` 이유 1: 라우터가 전달은 하지만 ICMP를 보내지 않는다. 뒤 홉은 다시 답한다.
- `* * *` 이유 2: 목적지 근처 방화벽이 탐침을 버린다. 뒤 홉도 답하지 않는다.
- 한 홉에서 +100 ms 정도 뛰면 해저 케이블이다. 측정: hop 7→8에서 +114 ms(MIT 경로).
- 애니캐스트 `1.1.1.1`: 13홉, 12 ms → 복제 서버가 국내(ICN)에 있다.

R6 OSPF 시간 측정 (FRR 라우터 3대)

사건	시간	이유
링크 끊기 (반송파 끊김)	< 0.4초	인터페이스가 바로 down → 새 LSA. Hello/Dead와 무관.
조용한 고장 (100% 손실)	38초	Dead 40초만 감지한다.
복구	6.2–18초	Hello, DB 교환 후 Full(+약 9초). 그 뒤 MinLSArrival(1초)로 버려진 LSA를 5초 뒤 재전송.
비용 변경 10→100	0.3초	바꾼 라우터는 바로 안다. 살아 있는 인접 관계로 LSA가 퍼진다.

요점: 계산보다 감지가 느리다. 끊김과 복구는 대칭이 아니다. 끊김은 사건 하나로 알 수 있다. 복구는 핸드셰이크와 DB 동기화가 필요하다.

R7 실습 Task 3 · 증분 SPF

- 사건 사이에 `dist`(출발지부터 비용)를 보관한다. 표 = dist와 그래프의 함수다.
- 최단 경로에 없는 링크의 down/비용 증가 → 무시한다.
- down/비용 증가지만 노드에 다른 최단 진입로가 있다 → SPF 없이 다음 홉만 다시 계산한다.
- up/비용 감소인데 더 짧아지지 않는다($d_a + c \geq d_b$) → 무시한다. 짧아지면 그 노드부터만 완화한다.
- 전체 SPF가 필요한 경우는 하나: 노드가 **유일한 최단 진입로를 잃을 때**.
- 결과 177회 대 1001회(82%). 파이썬에서는 실행 시간이 줄지 않는다. 라우터에서 부족한 것은 메모리가 아니라 control plane CPU다. CPU가 바쁘면 hello를 놓친다 → 플랩이 늘어난다.

E · 7주차 · 이더넷, ARP, 스위치 § 6.3–6.4

E1 MAC과 프레임

- MAC = 48비트. 계층이 없다(flat). NIC에 정해져 있다. 브로드캐스트 = `ff:ff:ff:ff:ff:ff`.
- IP = 어디에 있는가(계층형, 네트워크마다 바뀜). MAC = 누구인가(고정, 같은 링크 안에서만 씀).

```
| Preamble 8 | 목적지 MAC 6 | 출발지 MAC 6 | Type 2 |
Data 46–1500 | CRC 4 |
```

- 이더넷은 비연결형이고 비신뢰적이다. CRC가 틀린 프레임은 수신기가 버린다. ACK가 없다.
- 이더넷에는 TTL이 없다. 루프가 있으면 프레임이 끝없이 돈다 → STP를 쓴다.

E2 ARP (같은 서브넷에서 IP → MAC)

	요청 (request)	응답 (reply)
목적지 MAC	ff:ff:ff:ff:ff:ff (브로드캐스트)	요청자 MAC (유니캐스트)
받는 쪽	서브넷의 모든 호스트	요청자 하나
내용	"172.16.19.1 누구? …에게 알려 줘"	"172.16.19.1은 …에 있다"

- 요청에 요청자 MAC이 들어 있다. 그래서 응답은 브로드캐스트할 필요가 없다.
- ARP 표 = **소프트 상태**. 항목마다 만료 시간이 있다. 트래픽이 필요할 때 다시 생긴다.
- 목적지가 서브넷 밖이면 호스트는 서버가 아니라 **게이트웨이**를 ARP로 찾는다. 프레임 목적지 MAC = 게이트웨이, IP 목적지 = 서버.
- 그래서 먼 웹 서버는 ARP 표에 절대 없다. 표는 작게 유지된다.
- 측정: 비운 뒤 처음 돌아온 항목 = 게이트웨이 172.16.19.1(이전 2개, 이후 1개).
- 규모: 브로드캐스트 부하 $\propto$ 호스트 수. 호스트당 분당 1요청이면 /24 → 호스트마다 분당 254개, /16 → 초당 약 1,000개.
- 다른 서브넷으로 보내기: IP는 A→B 그대로. 프레임 1: 출발지 MAC A, 목적지 MAC 라우터. 라우터가 프레임 2를 만든다: 출발지 MAC 라우터 출력 포트, 목적지 MAC B.

E4 링크 계층 기타

- 패리티: 1비트로 홀수 개 오류를 감지한다. 2차원 패리티는 1비트 오류를 고친다.
- 인터넷 체크섬: 16비트 1의 보수 합.
- CRC: 생성자 G(r+1비트)를 고른다. $D \cdot 2^r$ XOR R을 보낸다. R = $(D \cdot 2^r / G)$의 나머지(mod 2). r+1비트 미만의 연속 오류를 감지한다.
- 다중 접속: 채널 분할(TDMA, FDMA), 랜덤(ALOHA, CSMA/CD), 차례(폴링, 토큰).
- Slotted ALOHA 최대 효율 $1/e \approx 0.37$. Pure ALOHA $1/(2e) \approx 0.18$.
- CSMA/CD 백오프: m번째 충돌 뒤 K를 $\{0,\cdots,2^{\min(m,10)}-1\}$에서 고른다. K·512비트 시간 기다린다.
- VLAN 802.1Q: 4바이트 태그, 12비트 VLAN ID. 트렁크 포트는 여러 VLAN을 싣는다.
- STP: 스위치들이 신장 트리를 만들고 루프가 되는 포트를 막는다.

E3 학습 스위치 (Task 1)

1. **학습**: 출발지 MAC → 들어온 포트, 현재 시각.
2. 들어온 포트로는 **절대** 내보내지 않는다.
3. 목적지가 브로드캐스트 → 나머지 모든 포트.
4. 아는 목적지 → 그 포트 하나(그 포트가 들어온 포트면 아무 데도 안 보냄, **필터링**).
5. 모르는 목적지 → 나머지 모든 포트(**플러딩**).

- 장비가 옮기면 (포트, 마지막 관측 시각)을 덮어쓴다 → 바로 새 포트를 믿는다(R3).
- 에이징: 300초보다 오래된 항목을 지운다(R4). 같은 시각 정보를 반대로 읽는 것이다.
- 브로드캐스트 주소는 출발지로 학습하지 않는다.
- **왜 들어온 포트로 내보내지 않는가?** 그 너머의 목적지는 이미 프레임을 받았다. 그 너머 스위치는 출발지를 틀린 포트로 배운다. 두 스위치가 모르는 프레임을 서로 되돌리면 끝없이 돈다(TTL 없음).
- **왜 스위치는 플러딩하고 라우터는 버리는가?** MAC은 계층이 없다. "모른다" = "아직 못 배웠다"다. 플러딩해도 LAN 안에서 끝난다. 라우터 표는 라우팅 프로토콜이 채운다. 경로 없음 = 도달 불가다. 인터넷 전체로 플러딩할 수도 없다.
- **시간 기반 에이징의 비용**: 받기만 하는 장비는 잊힌다 → 그 장비로 가는 프레임이 플러딩된다. 조용히 옮긴 장비 → 만료 전까지 옛 포트로 가서 사라진다. 더 직접적인 방법: 포트 down 시 그 포트 항목 삭제, STP 토폴로지 변경 알림.
- 스위치 vs 라우터: 스위치 = 2계층, 설정 없이 동작, 필터링, 트리 토폴로지 필요. 라우터 = 3계층, 설정 필요, 어떤 토폴로지도 가능, 브로드캐스트를 격리.

E5 꽉 찬 표 (Task 3)

- 포트 48, 장비 400, 용량 64. 트래픽이 치우쳐 있다(소수 장비가 대부분을 보냄).
- baseline: **가장 먼저 들어온 항목**을 내보낸다(FIFO). 들어온 시각은 앞으로 필요할지와 상관없다.
- 내 방식: MAC마다 출발지·유니캐스트 목적지 등장 횟수를 **count-min sketch**(4×256)로 센다. 추정치가 가장 작은 항목을 내보낸다. 20,000번마다 모든 카운터를 반으로 줄인다.
- **학습 거절**: 새 출발지는 횟수가 내보낼 항목보다 클 때만 들어온다. 거절 3,599번.
- 결과: 낭비 전달 323,380 → 159,712(−51%). LRU만 쓰면 −32%. 누락 0.
- sketch를 쓰는 이유: 메모리가 고정된다. MAC마다 카운터를 두면 가짜 MAC마다 커진다.
- **MAC flooding 공격**: 가짜 출발지 MAC을 계속 바꿔 프레임을 보낸다. 진짜 항목이 표에서 밀려난다. 진짜 호스트로 가는 프레임이 공격자 포트로도 플러딩된다 → 공격자가 엿본다. 대책: **포트 보안**(포트당 MAC 수 제한).

C0 파이썬 함정 (먼저 읽기)

함정	올바른 형태
파이썬의 ~x는 음수다	`~x & 0xFFFFFFFF`
p = 0일 때 마스크	`(0xFFFFFFFF<<32)&0xFFFFFFFF` = 0 ✓ (C에서는 32비트 시프트가 정의되지 않음)
반복 중에 리스트를 바꿈	복사본으로 돈다: `for m in list(d):`
키를 선형 탐색	사전(dict)을 쓴다: O(1)
처음 값 유지	`d.setdefault(k, v)`
FIFO 내보내기 / LRU	`od.popitem(last=False)`; `od.move_to_end(k)`
heapq 동점	튜플 `(cost, name, node)`을 넣는다. 튜플은 왼쪽부터 비교한다.
힙의 낡은 항목	`if node in done: continue`
나눗셈 / 시프트	`x >> 8` = x // 256
정수에서 바이트 꺼내기	`(v >> 24) & 0xFF`
문자열 → 정수	`int(s, 16)`; 옥텟은 `s.isdigit()` 확인

C1 IPv4 문자열 ↔ 정수 (ipaddress 금지)

```
def to_int(a):
    p = a.split(".")
    if len(p) != 4: raise ValueError(a)
    v = 0
    for x in p:
        if not x.isdigit() or int(x) > 255: raise ValueError(a)
        v = (v << 8) | int(x)
    return v

def to_str(v):
    return ".".join(str((v >> s) & 0xFF) for s in (24, 16, 8, 0))

def mask(p):
    return (0xFFFFFFFF << (32 - p)) & 0xFFFFFFFF
```

C6 parse_cidr, network_range, LPM 표

```
def parse_cidr(c):
    addr, pl = c.split("/")
    p = int(pl)
    if not 0 <= p <= 32: raise ValueError("prefix")      # R1
    net = to_int(addr)
    if net & ~mask(p) & 0xFFFFFFFF: raise ValueError("host bits")  # R2
    return net, p

def network_range(c):
    net, p = parse_cidr(c)
    last = net | (~mask(p) & 0xFFFFFFFF)
    if p == 32: return to_str(net), to_str(net), None
    if p == 31: return to_str(net), to_str(last), None
    return to_str(net+1), to_str(last-1), to_str(last)

class ForwardingTable:
    def __init__(self): self.by_len = {}       # p -> {net: hop}
    def add(self, c, hop):
        net, p = parse_cidr(c)
        b = self.by_len.setdefault(p, {})
        if net in b and b[net] != hop: raise ValueError("모호함")
        b[net] = hop
    def lookup(self, a):
        x = to_int(a)
        for p in sorted(self.by_len, reverse=True):   # 긴 길이부터
            h = self.by_len[p].get(x & mask(p))
            if h is not None: return h                # 첫 일치에서 멈춤
        return None
```

C7 빠른 LPM: 비트 트라이와 DIR-24-8

```
class Trie:                          # 작업량 <= 32단계
    def __init__(self): self.root = [None, None, None]  # [0-자식, 1-자식, hop]
    def add(self, net, p, hop):
        n = self.root
        for i in range(p):
            b = (net >> (31 - i)) & 1
            if n[b] is None: n[b] = [None, None, None]
            n = n[b]
        n[2] = hop
    def lookup(self, a):
        n, best = self.root, self.root[2]
        for i in range(32):
            n = n[(a >> (31 - i)) & 1]
            if n is None: break
            if n[2] is not None: best = n[2]   # 지금까지 가장 긴 일치 기억
        return best

# DIR-24-8: first = 2**24칸 배열 (/24마다 hop 인덱스 1개)
# add(p<=24): [net>>8, (net>>8) + 2**(24-p)) 칸을 채운다.
#   그다음 그 범위 안의 더 긴 접두사를 짧은 것부터 다시 채운다.
# lookup(a): return hops[first[a >> 8]]   (p>24는 작은 dict를 먼저 확인)
```

C2 반복 리졸버 루프 (3주차 T1)

```
def walk(name):
    servers = ROOTS
    for _ in range(MAX_DEPTH):                 # R6 깊이 제한
        reply = first_reply(servers, name)     # R4: 응답 없으면 다음 서버
        ans = reply.answer
        if name의 A 레코드가 ans에 있음: return ("A", ip)
        if ans에 CNAME 있음: return ("CNAME", target)   # R5: 루트부터 다시
        ns = reply.authority의 NS 이름들
        glue = [reply.additional에서 ns의 A]
        servers = glue if glue else [resolve(ns[0])]  # R3: glue 없음 = 새 탐색
    raise RuntimeError("depth")

def resolve(name):
    for _ in range(MAX_CNAME):
        kind, v = walk(name)
        if kind == "A": return v
        name = v
    raise RuntimeError("CNAME loop")
```

전송: `dig @SERVER NAME A +norecurse +time=3 +tries=1`. `status:`와 각 섹션을 읽는다.

C3 올바른 TTL 캐시 (3주차 T3)

```
class YourCache:
    def __init__(self, upstream):
        self.upstream = upstream; self.entries = {}    # name -> (addr, expires)
    def lookup(self, name, now):
        e = self.entries.get(name)
        if e is not None and now < e[1]: return e[0]  # TTL 안
        addr, ttl = self.upstream(name)               # 없거나 만료
        self.entries[name] = (addr, now + ttl)
        return addr
```

baseline 버그: 60초 고정(TTL 무시) → stale + 불필요한 재요청. 리스트 선형 탐색. 반복 중 `list.remove`.

C4 Selective repeat (4주차 T1)

```
# Sender.step()
pkt = acks.receive()
if pkt and pkt[0] == "A" and 0 <= pkt[1] < N:
    acked[pkt[1]] = True                # 중복 ACK: 문제없음
while base < N and acked[base]: base += 1   # 윈도우 밀기
if base == N: return False              # 끝
for seq in range(base, min(base + WINDOW, N)):
    if acked[seq]: continue
    if sent_at[seq] is None or now - sent_at[seq] >= TIMEOUT:
        sent_at[seq] = now
        out.send(("D", seq, chunks[seq])); break   # 스텝당 1번 전송
return True

# Receiver.step()
pkt = inp.receive()
if pkt is None or pkt[0] != "D": return
_, seq, data = pkt
acks.send(("A", seq))                   # 모든 도착에 ACK, 중복도
if seq >= next: buf.setdefault(seq, data) # 중복은 아무것도 바꾸지 않음
while next in buf:
    out += buf.pop(next); next += 1     # 순서대로 전달
```

GBN 변형: 수신기는 next만 보관한다. seq == next만 받고 나머지는 버린다. 항상 ACK(next−1)을 보낸다. 송신기: 타이머 1개. 타임아웃이면 base..base+N−1을 다시 보낸다.

Stop-and-wait (rdt3.0): 1비트 순번(0/1). 보내고 타이머를 켠다. 같은 비트의 ACK를 기다린다. 아니면 다시 보낸다.

C5 혼잡 제어: slow start + AIMD (4주차 T3)

```
class YourControl:
    def __init__(self):
        self.window = 1.0; self.slow_start = True; self.quiet = 0
    def on_ack(self):
        if self.quiet > 0: self.quiet -= 1
        if self.slow_start: self.window += 1             # RTT마다 2배
        else: self.window += 0.25 / self.window          # RTT마다 +0.25
    def on_loss(self):
        if self.quiet > 0: return          # 손실 묶음 1개 = 사건 1개 = 감소 1번
        self.slow_start = False
        self.window = max(2.0, self.window * 0.5)
        self.quiet = int(self.window) * 3
```

ssthresh를 쓰는 Reno: `if cwnd < ssthresh: cwnd += 1 else: cwnd += 1/cwnd`. 중복 ACK 3개: `ssthresh = cwnd/2; cwnd = ssthresh + 3`. 타임아웃: `ssthresh = cwnd/2; cwnd = 1`.

C8 Dijkstra + 첫 홉 (6주차 T1)

```
import heapq
def dijkstra(g, s):
    dist = {s: 0}; pq = [(0, s)]; done = set()
    while pq:
        c, n = heapq.heappop(pq)
        if n in done: continue
        done.add(n)
        for v, w in g[n].items():
            if v not in done and c + w < dist.get(v, float("inf")):
                dist[v] = c + w; heapq.heappush(pq, (c + w, v))
    del dist[s]; return dist                  # 도달 불가 노드는 없음

def forwarding_table(g, s):
    best = {s: (0, "")}; pq = [(0, "", s)]; done = set(); t = {}
    while pq:
        c, hop, n = heapq.heappop(pq)
        if n in done: continue
        done.add(n)
        if n != s: t[n] = hop
        for v, w in g[n].items():
            if v in done: continue
            cand = (c + w, v if n == s else hop)   # 첫 홉을 물려받음
            if cand < best.get(v, (float("inf"), "")):  # 동점: 작은 홉 이름
                best[v] = cand; heapq.heappush(pq, (cand[0], cand[1], v))
    return t

def link_down(g, a, b):
    g = {n: dict(e) for n, e in g.items()}   # 복사, 원본 변경 금지
    g[a].pop(b, None); g[b].pop(a, None); return g
```

Bellman-Ford / DV: |V|−1번 반복한다. 모든 간선 (u,v,w)에 대해 `if d[u]+w < d[v]: d[v]=d[u]+w`.

C9 증분 SPF 판정 (6주차 T3)

```
tight(x→y): dist[x] + cost(x,y) == dist[y]
비용 감소/up:  da + c < db 이면 b부터 완화   (아니면 무시, 또는 동점만 재계산)
비용 증가/down: tight인 y가 유일한 tight 선행자를 잃으면: 전체 SPF
                아니고 링크가 tight였거나 tight가 되면: 다음 홉만 재계산 (SPF 없음)
                그 외: 아무것도 안 함
```

C10 학습 스위치 (7주차 T1)

```
def handle(self, frame, in_port):
    src, dst, now = frame
    self._age(now)                          # seen < now - 300 삭제
    if src != BROADCAST: self.entries[src] = (in_port, now)  # 학습 / 이동
    others = [p for p in range(self.ports) if p != in_port]
    if dst == BROADCAST or dst not in self.entries: return others  # 플러딩
    out = self.entries[dst][0]
    return [] if out == in_port else [out]           # 필터링 / 전달
```

C11 Count-min sketch (7주차 T3)

```
slots(mac) = [hash_r(mac) % WIDTH for r in range(ROWS)]  # 행마다 독립 해시
count(mac): 각 행 r에서 sketch[r][slot_r] += 1
estimate(mac) = min(sketch[r][slot_r] for r)   # 실제보다 작게 세지 않음
표가 꽉 참: victim = 추정치 최소 항목; est(src) > est(victim)일 때만 src 학습
```

X · 명령어와 도구 COMMANDS

X1 DNS

`dig @198.41.0.4 NAME +norecurse`	반복 질의 1번
`dig +short NAME A`	답만 출력
`dig +trace NAME`	전체 탐색 (3주차 T1에서 금지)
`dig NAME NS / MX / CNAME`	다른 타입
`dig @1.1.1.1 / @8.8.8.8 / @9.9.9.9`	리졸버 바꾸기

X2 캡처와 읽기

`tcpdump -i IF -w x.pcap port 53`	캡처
`tshark -r x.pcap -F pcapng -w x.pcapng`	형식 변환
`tshark -r f -Y "dns.flags.authoritative==1"`	표시 필터
`-Y "tcp.flags.syn==1"`	핸드셰이크
`-Y "dhcp"`, `-Y "arp"`	DORA, ARP

캡처 필터(BPF): `port 53`, `tcp port 443`, `port 67 or port 68`, `arp`.
표시 필터(Wireshark): `dns`, `tcp.analysis.retransmission`.

X3 실습 명령

```
python3 task1_*.py --verify     python3 bench.py --yours
python3 test_tasks.py --task N  python3 ../check.py w03
bash w06-routing/scenario.sh up|routes|cut|restore|cost|down
```

X4 주소, 경로, ARP

`ip addr` / `ifconfig` / `ipconfig`	인터페이스 주소, 마스크
`ip route` / `netstat -rn`	기본 게이트웨이
`curl ifconfig.me`	바깥에서 본 공인 주소
`arp -a`, `ip neigh`	ARP 표
`sudo ip -s -s neigh flush all`	ARP 표 비우기 (리눅스)
`traceroute HOST` / `tracert` / `tracepath`	경로, 홉별 RTT
`iperf3 -c HOST`	처리량
`tc qdisc add dev IF root netem delay 31ms`	지연 추가
`tc ... netem loss 100%`	조용한 고장
`vtysh -c "show ip route"`	FRR 경로
`vtysh -c "show ip ospf neighbor"`	OSPF 이웃

X5 측정 전 확인

- VPN·터널(WARP, Tailscale)을 끈다. 경로와 처리량이 바뀐다.
- 5번 잰다. 숫자 하나가 아니라 중앙값과 편차를 보고한다.
- 제출 전 MAC, 호스트 이름, 내 IPv6를 가린다(`mask.py`).